Python Exceptions

Difference Between Error and Exception

Error

- Errors are serious issues in the program that **cannot be handled at runtime**.
- They usually occur because of mistakes in the code's syntax or configuration.
- Errors must be **fixed by the programmer**.
- Examples: **SyntaxError, MemoryError**

◆ Example:

```
print("Hello world"   # ❌ Missing closing parenthesis → SyntaxError
```

Exception

- Exceptions are issues that occur **while the program is running**.
- They are **less severe than errors** and can be handled by the program.
- Examples: dividing by zero, invalid input, file not found, etc.
- Handling exceptions allows the program to **continue running** without crashing.

◆ Example:

```
n = 10

res = n / 0  # ❌ ZeroDivisionError (Exception)
```

Exception Handling in Python

Python provides try, except, else, and finally blocks to handle exceptions.

1. try-except Block

The code inside the try block is executed.
If an exception occurs, it jumps to the except block.

```
try:

   print(x)   # x is not defined → NameError

except:

   print("An exception occurred")
```

2. Handling Multiple Exceptions

You can define **specific exception types** in separate except blocks.

```
try:
    print(x)
except NameError:
    print("Variable x is not defined")
except:
    print("Something else went wrong")
```

3. try-except-else

- The else block runs **only if no exception occurs**.

```
try:
    print("Hello")
except:
    print("Something went wrong")
else:
    print("Nothing went wrong")
```

4. try-except-finally

- The finally block **always executes**, no matter what.
- Useful for cleanup operations like closing files or releasing resources.

```
try:
    print(x)  # x is not defined
except:
    print("Something went wrong")
finally:
    print("The 'try except' is finished")
```

5. try-except-else-finally Together

All four blocks can be combined.

```
try:
    n = int(input("Enter a value: "))
    res = 100 / n
except ZeroDivisionError:
    print("You can't divide by zero!")
except ValueError:
    print("Enter a valid number!")
else:
    print("Result is", res)
finally:
    print("Execution complete.")
```

6. Nested try Blocks

A try block can exist inside another try block.

```
try:
    file = open("C:/Automation/automationFiles/myfile.txt", "w")
    try:
        file.write("Welcome")
    except:
        print("Something went wrong when writing to the file")
    finally:
        file.close()
except:
    print("Something went wrong when opening the file")
```

Raising Exceptions

Sometimes you may want to **manually raise an exception** when certain conditions are not met.

1. raise Exception

```
x = -1
if x < 0:
    raise Exception("Sorry, no numbers below zero")
```

2. raise Specific Exception

You can raise a particular type of exception.

```
x = "hello"
if not type(x) is int:
    raise TypeError("Only integers are allowed")
```

User-Defined Exceptions

You can raise your own custom exceptions to handle specific situations.

```
def set(age):
    if age < 0:
        raise ValueError("Age cannot be negative.")
    print(f"Age set to {age}")

# usage
try:
    set(-5)
except ValueError as e:
    print(e)
```

Key Takeaways

1. **Errors** → Serious, must be fixed in code.
2. **Exceptions** → Runtime issues that can be handled.
3. Use **try-except** to catch exceptions.
4. **else** runs if no error occurs.
5. **finally** always runs (cleanup).
6. You can **raise exceptions** manually.
7. **User-defined exceptions** improve error handling in custom cases.